SVM and LR for wine classification

Lanza Luca

February 28, 2026

Contents

1 Introduction **2**

2 Dataset analysis **2**

3 Support Vector Machine and Logistic Regressor **3**
3.1 How the models work . . . 3
3.1.1 Implementation . . . 4
3.2 Performance . . . 4
3.2.1 Hyperparameters . . . 5

4 Cross validation **5**
4.1 Implementation . . . 5
4.2 Running the code . . . 6
4.3 Consideration on the hyperparameters choice . . . 6

5 Kernelization **6**
5.1 Kernel function . . . 6
5.2 Kernelized Model . . . 6
5.3 Performance of the model . . . 7

6 Evaluation **9**
6.1 How performances are calculated . . . 9
6.2 Confusion matrix . . . 10
6.3 Conclusion . . . 11
6.4 Possible optimization . . . 11

7 Declaration **11**

1 Introduction

In this short experiment we will analyze how to classify wine using two different models, a Support vector machine and a Logistic regressor.

2 Dataset analysis

The dataset is divided into red wines and white wines. Both types share 11 different numerical features that together compose the physicochemical profile of the samples. While all these features are recorded, it is not known if every single one is actually useful for classification. Along with the 11 physicochemical characteristics and the target variable (quality), there is an additional nominative feature called 'color' to distinguish between white and red wine. The dataset was used in its entirety and not divided by color (red wine and white wine) for classification purpose, and because of this, data points might show an high variance despite having the same quality value. The data that I used were processed and standardized by subtracting the mean and dividing by the standard deviation to get a value that usually is between -3 and 3:

$$Z = \frac{X - mean}{std}$$

This is necessary because the features often have very different scales and the training would be unbalanced otherwise. The output (quality of wine) was also transformed into a binary format (-1 or +1) converting all quality scores lower than 6 to -1 and the others to +1. During the data analysis I observed that the dataset is highly unbalanced, which led me to try two different data augmentation techniques:

- Undersampling: this is a common technique, where the data in excess towards a class (in this case the high quality data) are removed from the dataset until a balance is reached;
- SMOTE: Synthetic minority oversampling technique, this approach uses interpolation between two points to generate a brand new point.

Neither of these techniques significantly improved the models performance compared to the baseline unbalanced dataset. The former gave unsatisfying results due to lack of samples in the dataset. The latter reasonably created data sample which were of low precision, due to the fact that many positive and negative points were very similar (the majority of the wines belong to the quality 5 or 6) resulting in an overlapping of the two classes. It is highly probable that interpolating between two quality 5 wines produces synthetic features that are indistinguishable from a quality 6 wine, poisoning the dataset. Consequently, SMOTE proved to be unsuitable for this specific classification task. Furthermore, while most samples are concentrated in the medium quality range (5-6 quality) there is a severe absence of wines with a quality value at the edge of the scale (such as 3 or 9), as illustrated in the figure below:

```
•••  dataset: 6497 row
     quality
     6          2836
     5          2138
     7          1079
     4           216
     8           193
     3            30
     9             5
     Name: count, dtype: int64
```

Figure 1: Distribution of the quality of the data.

Finally, the data has been divided for training test, evaluation set and test set, respectively with a 70%, 15% and 15% ratio.

3 Support Vector Machine and Logistic Regressor

3.1 How the models work

The two models that are implemented in the experiment are a Support Vector Machine and a logistic regressor. The support vector machine operates by maximizing a margin. In binary classification the objective is to find the hyperplane that better separates the two different classes (in this specific problem the good quality and the bad quality wines). The margin must be as wide as possible, meaning the distance between the hyperplane and the closest points of each class (the support vectors) is maximized. During the training phase, whenever the model finds a point that is too close to the margin or even misclassified, it updates the weights to try shifting or rotate the hyperplane in such a way that the point falls in the correct part of the new margin or it is far enough from it. For this task, the hinge loss is used:

$$max(0, 1 - z)$$

Where z is calculated as:

$$z = y(wx + b)$$

This kind of loss heavily penalizes the model when a point is misclassified or is inside the margin. Only data points with a z value that is positive and greater than or equal to 1 (meaning they are correctly classified and far from the margin) have a loss of 0. The logistic regressor, on the other hand, predicts the probability that a point belong to a certain class, which is $P(y = 1|x)$. To do so, it takes a linear equation z (like in the SVM case) and uses the sigmoid function, which is:

$$Sigm(z) = \frac{1}{1 + e^{-z}}$$

This process gives a number that belongs to the interval (0,1). The model than makes a decision on how to classify the point based on the computed probability. In the experiment I use 0.5 as threshold. The loss function used is the logarithmic loss, defined as:

$$loss(z) = \log_2 1 + e^{-z},$$

which penalizes the model more if it is very confident about a wrong prediction.

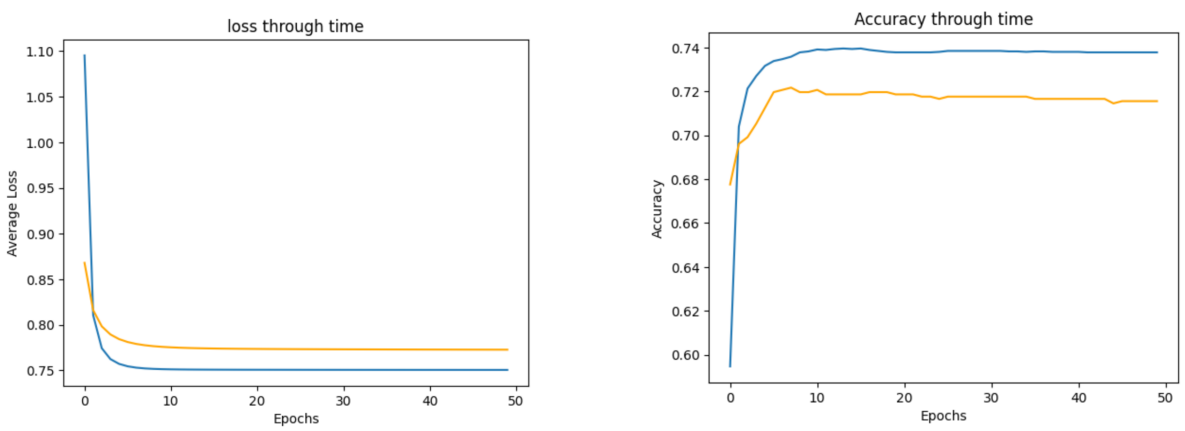
3.1.1 Implementation

The model is built in such a way to implement both the SVM and the LR with the same architecture. The training method gets a model parameter in input, which is a string that tells if the model should be a SVM (by writing "SVM") or a LR. After that, the model will then call the method `training_step` accordingly to the model used.

The training step of a support vector machine consists in calculating the product between the weights and the corresponding inputs and adding a bias, then calculating the loss (using the hinge loss) and the regularization parameter which will shrink the weights. The next step is the calculation of the gradient and the update of the weights, checking if the dot product is major or equal to 1. This tells us that the output and the weight are concordant, the product is a point that has enough distance from the hyperplane which divides the data, and the update only occurs to lower weights a little using the lambda regularization. Otherwise, the weights are updated by using the gradient, that is simply the partial derivative of $(1 - y(w^t x + b))$ with respect to w, resulting in $-yx$. The bias b is updated by taking the partial derivative with respect to b and so it is clear that in the SVM case $(1 - y(w^t x + b))$ the result is simply $-y$. The Logistic Regressor instead calculates the probability that the prediction done is correct. After calculating the prediction and the loss we can compute the probability $P(y|x)$ using the sigmoid function and the error $1 - P(y|x)$ which we will use to update the weights. The model uses the prediction method to make a prediction on the test set and check the results. In the SVM case, it is simply the dot product, while in logistic regression, the dot product is calculated to get the probability of output being 1. If the probability is over 0.5 it is classified as good wine, else as a bad wine.

3.2 Performance

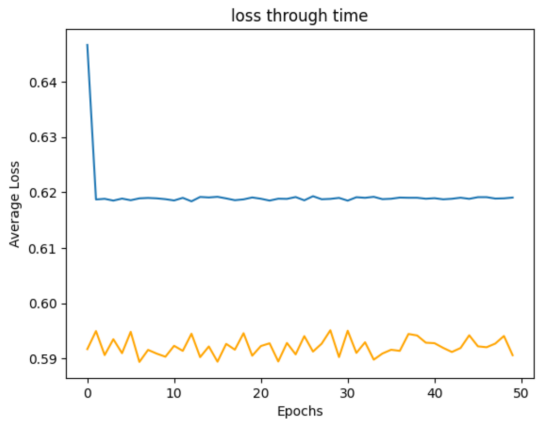
Here is a sample of both models performance:



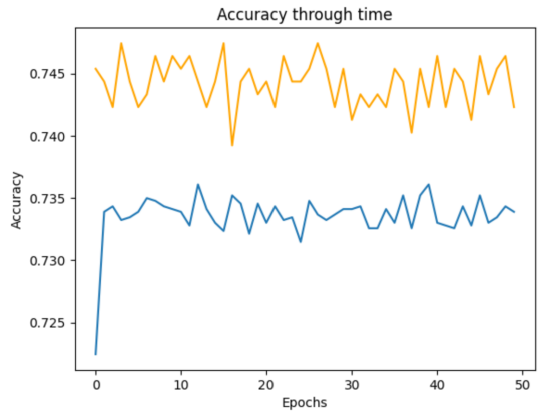
(a) Loss of the LR model

(b) Accuracy of the LR model

Figure 2: Loss and accuracy through the epochs for LR



(a) Loss of the SVM model



(b) Accuracy of the SVM model

Figure 3: Loss and accuracy through the epochs for SVM

Both keep an accuracy over the 70% while the loss used are different which is the reason why the loss is lower in SVM. It is interesting to note how the SVM loss and accuracy are unable to converge, thus showing a spike behavior, due to an excessive learning rate (which was 0.01). The model is underfitting due to an excessive lambda which blocks the growth of the weights, and the model cannot learn well.

3.2.1 Hyperparameters

The implementation has 3 hyperparameters: the epochs, the learning rate and the lambda regularization parameter.

- Learning rate: at each step, the update of each weight is decided using this parameter. A too high learning rate is responsible for a bad convergence of the model. Instead of reaching a minimum of the loss function, the model just bounces around it.
- Epoch: during an epoch every data point in the training set is used to train the model. Once every data point is computed and used to update the weights, the epoch ends.
- lambda: This is the regularization and is used to avoid overfitting. By regularizing the weights, we avoid that they become too large and so that a small change in the input provoke a large change in the output.

4 Cross validation

The hyperparameters used until now are randomly chosen, giving worse performance to the SVM model with respect to the LR one. Cross validation is used to optimize the hyperparameters choice.

4.1 Implementation

The cross validation works by dividing in k folds the set that is passed as an argument. In the implementation used, the division is in 5 folds: 4 of them are used in the training parts, while the other one is kept for validation.

This is repeated k times by swapping the fold used as validation each time, this way all the folds are used in training and in validation. Each time the model makes a prediction using the evaluation fold spared and calculates the performance metric, keeping track of the accuracy. After a single cross validation is done, the average accuracy is calculated. Then a grid search method is used in order to get all the possible combinations of the 3 hyperparameters that need to be tuned. For each computed average accuracy, the grid search saves the new hyperparameters as the best ones if they score better than the stored ones.

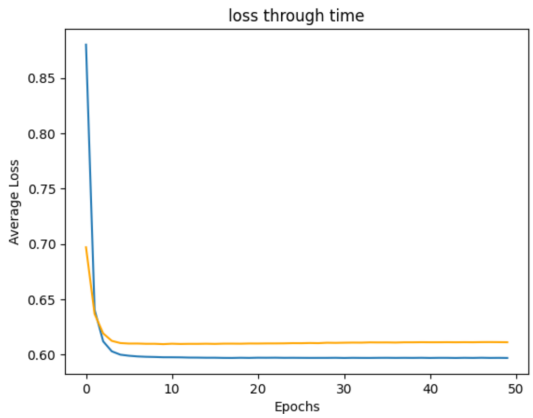
4.2 Running the code

After running the block that defines cross validation and grid search methods, run the 2 block cells below. The first one is used to fine tune the hyperparameters for a SVM model using only the training part of the data, and then test it with the best hyperparameters, found on all the dataset. The second block has the same function for a LR model.

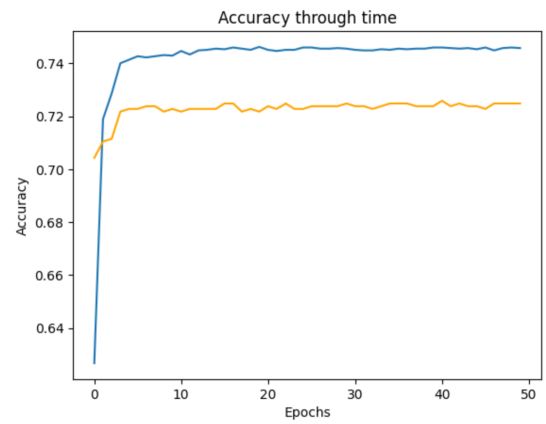
4.3 Consideration on the hyperparameters choice

The running and comparison of multiple hyperparameters confirm the right choice of the lambda parameter and learning rate in LR. Also in SVM the correct choice of learning rate and lambda should be 0.001 for both.

Applying again the training with the tuned parameter we obtain a much more pleasant result:



(a) Loss of the SVM model with tuned parameters



(b) Accuracy of the SVM model with tuned parameters

Figure 4: Loss and accuracy through the epochs for SVM

5 Kernelization

5.1 Kernel function

The purpose of the kernel function is to plot data in an higher dimensional space, to make it much easier to separate. I used two kernel functions for this experiment: the polynomial function, which maps an element from a R^d space to an higher dimension space given by the formula $k(x, x') = (c + x^t x')^d$, which represents another separation in the original space (like a parabola for d = 2). The Gaussian function instead uses the formula $e^{(-1/2\gamma)||x-x'||^2}$ which represents the distance between the points with a bigger penalty with respect to the polynomial function.

5.2 Kernelized Model

This model, like the base one, has both the SVM and the LR parts, which are declared while initializing the model by simply putting a string with the proper desired name. Additionally, it is necessary to specify the desired kernel function, between the polynomial and the gaussian function, and the required kernel parameters. The training starts with the computation of the matrix of similarity between all the points, which is symmetric by construction, by using the kernel trick and storing the results. Once the operation is completed (which means that all the distances of the training points in the higher dimensional space are computed), it is time to start the learning part.

First of all, the training step is calculated for each point. The objective is not to update the weights to understand how much a single feature counts, but understand how much a single point is important for the classification (meaning it can be very close to the margin). In SVM the dot product is calculated directly between the y and the row of K at step i, which is the similarity of the i-th point

with all the others. The loss is calculated by using the hinge loss, and the parameter (called alpha in the experiment) is updated. Notice that the alpha vector is first regularized to avoid exploding parameters and then, if the point was misclassified or inside the margin, the alpha value associated with the i-th point is updated by adding the learning rate, which means giving more importance to the point that was misclassified. The shrink of the regularization parameter helps to avoid the explosion of the alpha parameter.

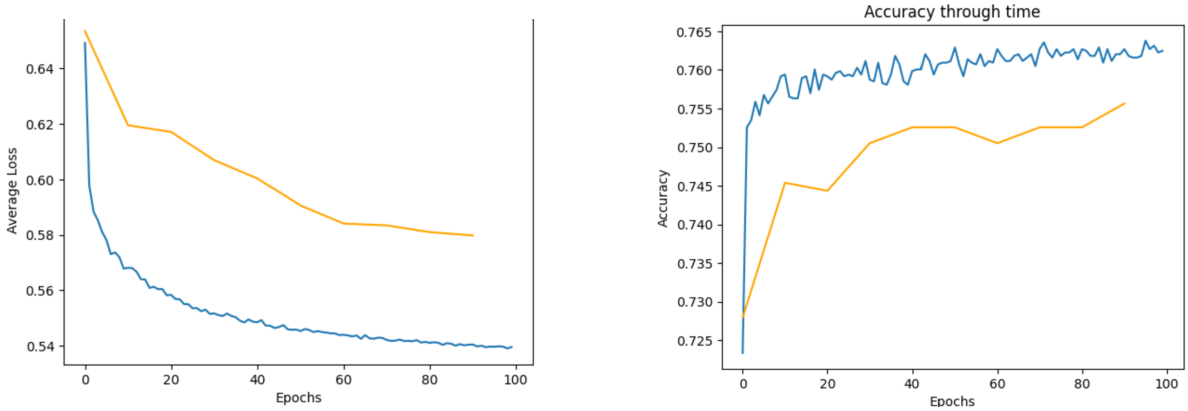
In the LR instead, after the calculation of the product as before, the exponential term and the sigmoid function are used and the probability of the output being 1 given x is calculated, in order to obtain the error, which is used to calculate the alpha update using the formula $alpha[i] = alpha[i] + learningrate * error$. As before, the regularization parameter is applied at each iteration to keep the alphas low.

After the training is completed, the model can be used with the prediction method on a set of points (which is my test set) to make new predictions and calculate the final performance of a model. The prediction method works by computing, for each new point inside the test set, the similarity with all the other points that belong to the train. The sum of all similarities is calculated, avoiding the alpha parameters, equal to 0, which represent points that have no relevance in the classification. For the SVM the point is then classified as the sign of the sum, while for the LR the classification is done by doing the same sum to obtain the value used to get the probability. Instead of doing so, it is better to calculate directly when the sigmoid function is greater than or equal to 0.5, condition met when z is greater than or equal to 0. Each prediction was then placed inside an array to evaluate if it was correct later. Finally, the evaluation method returns the loss and the accuracy of the model on the evaluation set. This works in SVM by simply calculating the sum of the inner product between the point to evaluate and the j-th point of the training set. As seen before, the cumulative sum of all the training points with respect to the evaluated point gives us our prediction. If the prediction is concordant with the expected output, the multiplication between these two values will be positive, and so it is classified correctly, increasing the accuracy of the model. The loss is then calculated with the hinge loss with respect to the prediction.

In LR, the evaluation is calculated in the same way, but the loss is the logarithmic loss, calculated using the sigmoid function.

5.3 Performance of the model

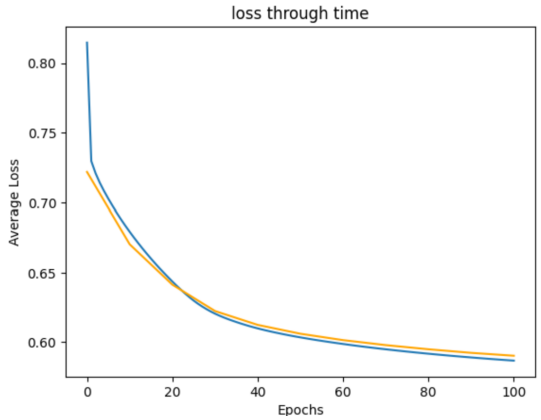
For each model the loss and accuracy are shown here graphically:



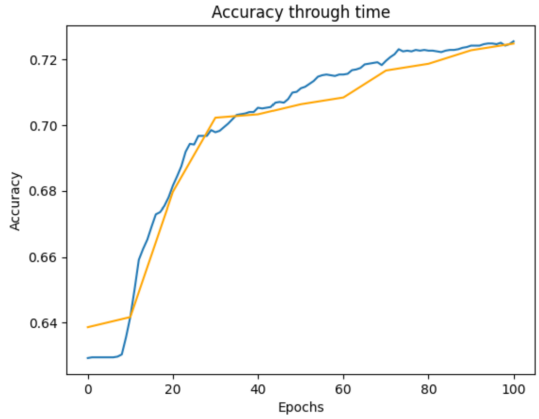
(a) Loss of the SVM polynomial model

(b) Accuracy of the SVM polynomial model

Figure 5: Loss and accuracy through the epochs for polynomial SVM

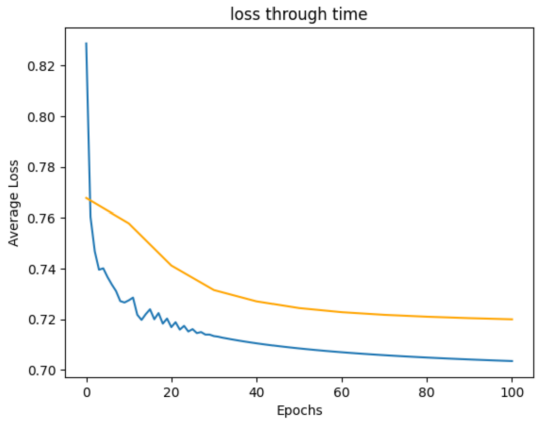


(a) Loss of the SVM gaussian model

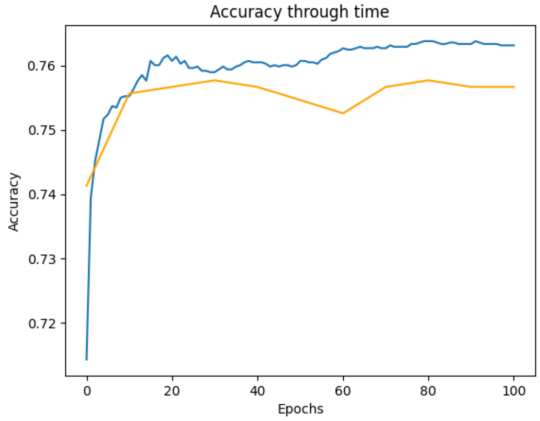


(b) Accuracy of the SVM gaussian model

Figure 6: Loss and accuracy through the epochs for SVM gaussian model

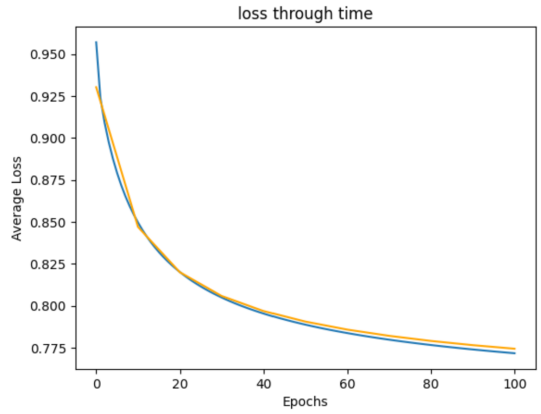


(a) Loss of the LR polynomial model

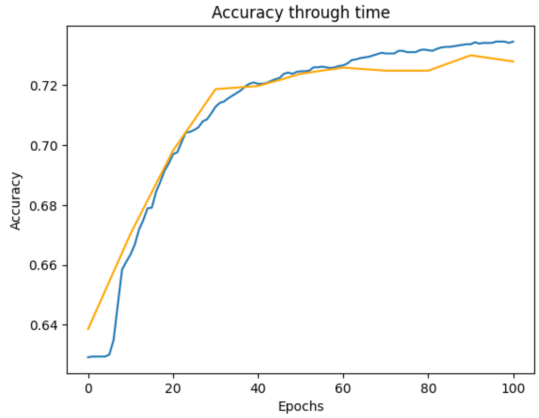


(b) Accuracy of the LR polynomial model

Figure 7: Loss and accuracy through the epochs for LR polynomial model



(a) Loss of the LR gaussian model



(b) Accuracy of the LR gaussian model

Figure 8: Loss and accuracy through the epochs for LR gaussian model

With respect to the standard SVM and LR model performance, we can see an increase of the accuracy, getting to 75%. It is important to note that the evaluation (in orange) is calculated as an average through 10 epochs for computational purpose.

A sample of the performance metric of the SVM polynomial model is shown here:

```
Accuracy: 0.7561475409836066
Precision: 0.7985074626865671
Recall: 0.8385579937304075
F1 score: 0.8180428134556575
```

Figure 9: Performance of the SVM polynomial model

The high recall give an important information about the dataset, showing that it is unbalanced, which is the reason why the model struggles more to correctly classify bad wines.

6 Evaluation

6.1 How performances are calculated

During the evaluation we can encounter four cases: the data is correctly classified as +1 (True positive), it is correctly classified as -1 (True Negative), it is incorrectly classified as +1 (False positive) or it is classified incorrectly as -1(False negative). Knowing this the performance metric used in the experiment are:

- accuracy. The accuracy is defined as the number of correct predictions overall, which is:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- precision. The precision measures the number of correct positive predictions on all the positive predictions that has been done, or

$$\frac{TP}{TP + TN}$$

- recall. The recall measures how much positive examples the model correctly identifies with respect to all the positive cases

$$\frac{TP}{TP + FN}$$

- f1 score. It is a metric useful for the unbalanced dataset, because it averages precision and recall, which are often very different in unbalanced dataset

$$2 * \frac{precision \times recall}{precision + recall}$$

The accuracy was used as the major performance metric to check the behavior of the models. Still the recall helped a lot in identifying the unbalance inside the dataset. The performance for both the SVM and LR model are here reported:

```
SVM test:
Accuracy: 0.7448770491803278
Precision: 0.7630890052356021
Recall: 0.8955453149001537
F1 score: 0.8240282685512368


Logistic regression test:
Accuracy: 0.7407786885245902
Precision: 0.7926470588235294
Recall: 0.8279569892473119
F1 score: 0.8099173553719008
```

Figure 10: Performance of the SVM and LR models

As already reported, the high recall tells us that the model is very good at catching the positive case, but it struggles with negative cases due to few example of negative case with respect to positive ones, which makes the model take the positive case more often in condition of uncertainty.

6.2 Confusion matrix

Another proof of the difficulty of the models with the bad wines come from the confusion matrix. This matrix has on the diagonal the correct classification, while the other elements represent the points that were incorrectly predicted. Here is shown the confusion matrix of the SVM model and how it performs in the prediction of the test set:

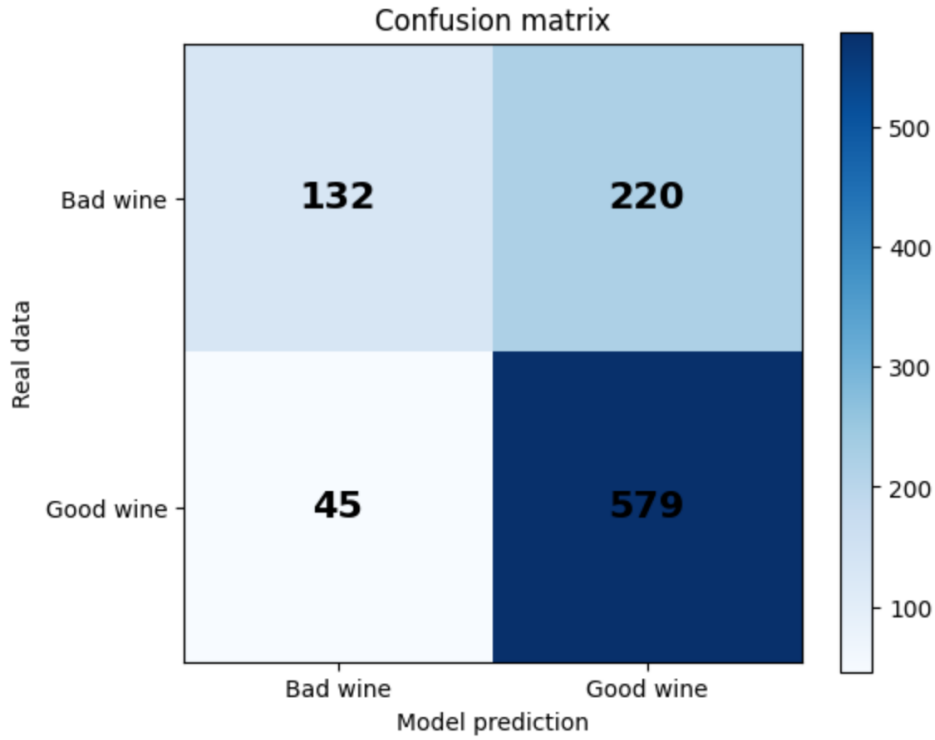


Figure 11: Confusion matrix of the SVM model

It is clear that when the model has to recognize a good wine it performs well, but it is still hard to manage the bad ones.

6.3 Conclusion

The models tested in the experiment works with some limits and, even if the data are not perfectly linearly separable, the models are able to adapt and develop a good accuracy. The correct identification of many bad wines shows that the models are not just answering "good wine" because the majority of the data are good wines. The underfit in the SVM model shows how much a correct choice of hyperparameter is required to get a correct convergence of the model. In this case, the cross validation really helped by tuning correctly the hyperparameter. With this dataset the model is still limited by the unbalancing, and from the mix of two different color of wines.

6.4 Possible optimization

Some possible strategy to increase performance could be:

- Add new bad wines to the dataset: because the dataset is unbalanced towards the good wines, it is beneficial to add some negative wines to balance it. This should reduce the bias towards good wines.
- Add more edge case wines: most of the wines in the dataset are between quality 5 or 6, making them probably very similar. Adding more low quality and high quality wines could help the model understand how some features affect wines and the quality.
- Divide the dataset: divide the dataset in two different kind of wines, red wine and white wines.

7 Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and

acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.